

CSN6214

Operating Systems

Trimester 2620

Advanced Concurrency & IPC Assignment

Topic: Multithreading, Multiprocessing, Inter-Process Communication (IPC), and System Calls

Language: C (POSIX Standard)

Total Marks: 100 (converted to 30% of coursework mark)

Due Date: [3rd October 2026 – 5:00 PM]

1. Objective

The objective of this assignment is to provide hands-on experience with advanced concurrency models, process synchronization, Inter-Process Communication (IPC), and low-level system calls using **C and POSIX APIs**. Students will analyze the performance trade-offs between multithreading (pthreads) and multiprocessing (fork) by implementing a **Parallel Merge Sort** system on a massive dataset, documenting the architectural decisions, and visually analyzing the execution times.

2. Group Formation & Contribution Policy

This assignment is a collaborative effort and must be completed under the following guidelines:

- **Group Size:** The assignment must be completed in a group of exactly 3 or 4 members.
- **Section Constraint:** All group members must be enrolled in the same lecture section.
- **Detailed Contribution Statement:** The group must submit a detailed contribution statement outlining exactly what each member did for both the code implementation and the technical report.
- **Report Inclusion:** This contribution statement must be explicitly and clearly stated in the Technical Report (e.g., as a dedicated section at the end of the report or in an appendix).
- **Individual Grading:** The final grade for the assignment is not automatically identical for all group members. Grades of individual members might be different based on their contributions.

3. Problem Statement

You are tasked with implementing a **Parallel Merge Sort System in C**. You can reference (https://en.wikipedia.org/wiki/Merge_sort) to learn about the merge sort.

3.1 Parallelization Strategy

Merge sort is a divide-and-conquer algorithm that is inherently parallelizable. Your implementation must follow this general approach:

1. **Divide:** Split the unsorted dataset into N contiguous chunks, where N is the number of worker threads or processes.
2. **Conquer (Parallel):** Each worker independently sorts its assigned chunk using merge sort.
3. **Merge (Synchronized):** Once all workers have finished sorting their chunks, perform a final N -way merge to produce the fully sorted dataset.

3.2 The Dataset & File I/O (System Calls)

You must generate a **single dataset of 250 million random integers** (approx. 1 GB). Crucially, you must store this data to disk and load it back into memory using **low-level POSIX system calls**.

1. **Data Generation & Storage:** Generate the 250 million integers (4 bytes each) and write them to a binary file on disk using the `write()` system call.
2. **Data Loading:** Read the data from the binary file back into your POSIX Shared Memory segment using the `read()` system call.

Constraint: You **must not** use standard C library buffered I/O functions (e.g., `fopen`, `fread`, `fwrite`, `fclose`). You must use the unbuffered system calls `open()`, `read()`, `write()`, and `close()` from `<unistd.h>` and `<fcntl.h>`.

3.3 Concurrency & OS Requirements (POSIX C)

Your implementation **must** utilize both multithreading and multiprocessing paradigms.

1. **Multithreading:** Use `<pthread.h>` to spawn N threads. Each thread sorts its assigned chunk of the array in-place. Use a **barrier** (`pthread_barrier_t`) or **mutex + condition variable** to synchronize the transition from the sort phase to the merge phase.
2. **Multiprocessing:** Use `fork()` and `<sys/wait.h>` to spawn N child processes. Each child sorts its assigned chunk.
3. **Process Synchronization:** You must use **Mutexes and/or Semaphores** to coordinate the merge phase.
 - *Examples:* Use `sem_open()/sem_wait()/sem_post()` so that the master process knows when all child processes have finished sorting. Use a `pthread_mutex_t` (placed in shared memory with `PTHREAD_PROCESS_SHARED`) to protect shared state during the merge phase.
4. **Inter-Process Communication (IPC):** You must implement **one** of the following for the multiprocessing model:
 - **Shared Memory:** Use POSIX shared memory (`shm_open`, `mmap` from `<sys/mman.h>`) to place the dataset in a shared memory region. All child processes must read/write directly to this shared region.

- **Message Passing:** Use POSIX Message Queues (`mq_open` from `<mqueue.h>`) or Pipes (`pipe()`) for the master process to dispatch chunk boundaries (start index, end index) to each child, and for children to send back completion status messages.

4. Experimental Design & Benchmarking

You must measure the **wall-clock execution time** (using `clock_gettime(CLOCK_MONOTONIC, ...)`) for the **sorting phase** (starting *after* the data has been successfully loaded into shared memory via `read()`).

For the **1 billion integer dataset**, run the following experiments:

1. **Multithreading:** Execute the parallel merge sort using 1 thread, 2 threads, 3 threads ... up to 10 threads.
2. **Multiprocessing:** Execute the parallel merge sort using 1 process, 2 processes, 3 processes ... up to 10 processes.

Ensure you run each configuration at least 3 times and take the average. Re-randomize and re-write/re-read the dataset before each run to avoid caching and sorting already-sorted data.

5. Required Visualizations

Your report must include the following three charts. (*Output your timing data to a CSV file and use Excel to generate these*).

Chart 1: Multithreading Performance - *Type:* Line Chart - *X-axis:* Number of Threads (1 to 10) - *Y-axis:* Execution Time (seconds) - *Data:* A single line representing the execution time of the 1 billion integer dataset.

Chart 2: Multiprocessing Performance - *Type:* Line Chart - *X-axis:* Number of Processes (1 to 10) - *Y-axis:* Execution Time (seconds) - *Data:* A single line representing the execution time of the 1 billion integer dataset.

Chart 3: Threads vs. Processes Comparison - *Type:* Comparative Line Chart (both lines on the same graph) - *X-axis:* Number of Workers (1 to 10) - *Y-axis:* Execution Time (seconds) - *Series:* “Threads” vs “Processes”.

6. Deliverables

6.1 Source Code & Build System (30 Marks)

- Well-documented **C code**. Must include clear comments explaining the POSIX API calls used for IPC, Mutexes, Semaphores, Shared Memory, and **File System Calls**.
- A **Makefile** that cleanly compiles the project. It must include targets for all, clean, and run.
- Include a README.md with instructions on required libraries and execution.

6.2 Technical Report (40 Marks)

A formal PDF report (max 15 pages) containing:

1. **Introduction & Architecture:** High-level design of your system. Include a block diagram showing the master/worker model, file I/O phase, parallel sort phase, and merge phase.
2. **Implementation Details:**
 - **System Calls:** How did you implement open, read, and write? How did you handle reading/writing a 4GB file in chunks? Discuss the difference between these system calls and standard C library I/O.
 - **IPC & Sync:** How did you implement mmap vs mq_open/pipes? How are Mutexes/Semaphores utilized? Provide C code snippets of the critical sections.
 - **Merge Strategy:** How does your N -way merge work?
3. **Experimental Results:** Include the 3 required charts.
4. **Analysis & Discussion:**
 - Why does execution time scale the way it does? At what point do you observe diminishing returns?
 - Apply **Amdahl's Law** to your results.
5. **Conclusion:** Summary of findings.

6.3 Demo Video (10 Marks)

- A screen-recorded video (max 5 minutes).
- Must show the C code structure, the Make file, briefly explain the architecture, and show the program executing.
- Show the terminal output (including file creation, data loading, correctness verification, and CSV generation).
- *Format:* MP4 or a shared link.

6.4 Experiment Data Log (10 Marks)

- Submit a raw_data.csv file containing the exact timing outputs generated by your C program across all thread/process counts.

7. Grading Rubric

C Code & POSIX APIs (25 Marks)

Sub-Criterion	0 Marks (Not Attempted)	2 Marks (Basic/Flawed)	3 Marks (Competent)	5 Marks (Excellent)
1.1 Multithreading (pthreads)	No thread implementation present.	Threads created but no synchronization; race conditions present; threads don't terminate cleanly.	Threads correctly sort chunks; uses 'pthread_barrier_t' or mutex + condvar to sync merge phase. Minor style issues.	Clean thread pool; proper barrier sync; no race conditions; threads join gracefully; error handling on all pthread_* calls.
1.2 Multiprocessing ('fork')	No fork() implementation present.	Children spawned but no waitpid(); zombie processes; no	Correct fork()/waitpid() loop; children terminate properly; basic error handling.	Robust fork logic; handles fork() failures; uses waitpid() with status checks; no

		error handling on fork.		zombies/orphans; signal-safe.
1.3 Shared Memory (shm_open + mmap)	No shared memory used; data duplicated per process.	mmap used but with MAP_PRIVATE (defeats purpose); or shm_open fails silently.	shm_open + mmap with MAP_SHARED correctly used; dataset shared across processes.	Full POSIX shm lifecycle (shm_open, ftruncate, mmap, munmap, shm_unlink); error handling on every call; no memory leaks.
1.4 Message Passing (MQ or Pipes)	No message passing implemented.	Partial implementation; messages sent but not received; blocks indefinitely; no cleanup.	Master dispatches chunk boundaries; children receive and acknowledge; basic pipe or mq_open usage.	Bidirectional communication; master sends tasks, children return status + stats; proper mq_close/mq_unlink or pipe cleanup; non-blocking where appropriate.
1.5 Synchronization (Mutex/Semaphore)	No mutexes or semaphores used.	Sync primitives declared but not used correctly; deadlocks occur; missing unlocks.	Mutex/semaphore protects critical sections (e.g., shared counter, merge phase); sem_open/pthread_mutex_init with PTHREAD_PROCESS_SHARED where needed.	Robust locking strategy; no deadlocks; semaphores coordinate merge phase; mutex attributes set correctly for cross-process use; sem_unlink/pthread_mutex_destroy called.
1.6 File I/O System Calls	Uses fopen/fread/fwrite (forbidden).	Uses open/read/write but only single-call (fails on 4GB); no chunking loop.	Uses open/read/write/close with a chunking loop (e.g., 4–16 MB chunks); handles partial reads/writes.	Chunked I/O with full error handling; checks return values; handles EINTR; uses O_CREAT/O_WRONLY/O_TRUNC flags correctly; -D_FILE_OFFSET_BITS=64 applied.

Report Quality (25 Marks)

Sub-Criterion	0 Marks (Not Attempted)	2 Marks (Basic/Flawed)	3 Marks (Competent)	5 Marks (Excellent)
2.1 Introduction & Architecture Diagram	Missing or irrelevant.	Diagram present but unclear; missing key components (master, workers, IPC).	Clear block diagram showing master/worker model, file I/O, sort phase, merge phase, and IPC channels.	Professional diagram with labeled data/control flows; clearly distinguishes shared memory vs message passing paths; includes synchronization points.

2.2 System Call Explanation	No discussion of open/read/write.	Mentions system calls but doesn't explain chunking or difference from stdio.	Explains why chunking is needed; contrasts system calls with fread/fwrite (kernel vs user-space buffering).	Deep analysis: discusses context-switch overhead per syscall, kernel buffer pages, 'O_DIRECT' tradeoffs, and why unbuffered I/O was chosen.
2.3 IPC Implementation Details	No IPC discussion.	Lists APIs used without explanation of why or how.	Explains mmap vs mq_open design choices; includes relevant code snippets.	Critical comparison of shared memory vs message passing tradeoffs; code snippets annotated with OS-level behavior (page faults, copy-on-write avoidance).
2.4 Synchronization Strategy	No sync discussion.	Mentions mutex/semaphore without explaining critical sections.	Identifies critical sections; shows code snippets; explains locking order.	Justifies choice of semaphore vs mutex; discusses deadlock prevention; shows PTHREAD_PROCESS_SHARED setup; analyzes lock contention.
2.5 Merge Strategy Explanation	Merge phase not described.	Mentions merge but no detail on N-way algorithm.	Describes sequential or parallel N-way merge; explains complexity.	Analyzes merge as the sequential bottleneck; discusses k-way merge using min-heap; proposes optimizations (parallel merge tree).
2.6 Experimental Results (Charts)	Charts missing.	Charts present but mislabeled, wrong axes, or don't match raw data.	All 3 charts present, correctly labeled, match CSV data.	Charts are publication-quality; include error bars (from 3 runs); clear legends; insightful axis scaling; trends are visually obvious.
2.7 Analysis & Amdahl's Law	No analysis.	Superficial observations ("more threads = faster").	Applies Amdahl's Law; identifies sequential fraction; discusses diminishing returns.	Quantifies sequential fractions from data; validates against theoretical speedup curve; discusses memory bandwidth saturation and cache effects.
2.8 Writing Quality & Formatting	Unreadable or missing.	Poor grammar; inconsistent formatting; no section headers.	Clear writing; consistent formatting; proper sectioning; citations if any.	Professional academic tone; polished formatting; logical flow; executive summary; well-structured conclusion.

Visualizations (10 Marks - 2 sub-criteria × 5 marks)

Sub-Criterion	0 Marks (Not Attempted)	2 Marks (Basic/Flawed)	3 Marks (Competent)	5 Marks (Excellent)
3.1 Chart Correctness & Completeness	Charts missing entirely.	Only 1–2 charts present; data doesn't match CSV; axes wrong.	All 3 required charts present; data matches raw_data.csv; correct chart types (line/comparative).	All 3 charts present with correct types; includes markers at each data point; gridlines for readability; units clearly labeled.
3.2 Chart Clarity & Interpretability	Charts unreadable or misleading.	Charts present but cluttered; no legend; poor color choices; hard to distinguish lines.	Legends present; colors distinguishable; titles descriptive.	Colorblind-friendly palette; annotations highlighting key insights (e.g., "diminishing returns at N=6"); professional styling.

Demo Video (10 Marks - 2 sub-criteria × 5 marks)

Sub-Criterion	0 Marks (Not Attempted)	2 Marks (Basic/Flawed)	3 Marks (Competent)	5 Marks (Excellent)
4.1 Content Coverage	No video submitted.	Video shows only compilation OR only execution; skips architecture explanation.	Shows code structure, Makefile, architecture explanation, compilation, execution, and graph generation.	Covers all required elements plus correctness verification (PASS output), CSV generation, and brief discussion of interesting observations.
4.2 Production Quality	Unwatchable (no audio, corrupted, >10 min).	Poor audio/video; rushed; no narration; exceeds 5 min significantly.	Clear audio and video; under 5 minutes; logical flow; narration present.	Professional narration; screen zooms on key code sections; well-paced; engaging; demonstrates deep understanding verbally.

Data & Execution (10 Marks - 2 sub-criteria × 5 marks)

Sub-Criterion	0 Marks (Not Attempted)	2 Marks (Basic/Flawed)	3 Marks (Competent)	5 Marks (Excellent)
5.1 Raw Data Log (raw_data.csv)	No CSV file submitted.	CSV present but incomplete (missing thread/process counts); data doesn't match report charts.	CSV contains all 20 data points (1–10 threads + 1–10 processes) with timing values matching report.	CSV includes 3 runs per configuration (60 rows); includes standard deviation column; well-formatted with headers; matches charts exactly.
5.2 Evidence of Full Execution	No evidence that 1-billion dataset was actually tested.	Evidence suggests only small datasets were tested; timings	Terminal output shows 1-billion dataset loaded; timings plausible	Timestamped logs prove full execution; correctness 'PASS' shown for all runs; resource usage (e.g., time

		inconsistent with 1GB workload.	(~seconds to minutes per run).	output) included; reproducible via make run.
--	--	---------------------------------	--------------------------------	--

Individual Assessment & Peer Evaluation (20 Marks)

Sub-Criterion	0 Marks (Not Attempted)	2 Marks (Basic/Flawed)	3 Marks (Competent)	5 Marks (Excellent)
6.1 Code Ownership & Quality	Member contributed no code, or code is entirely copied/unused.	Member contributed only trivial code (e.g., simple print statements, basic variable declarations); cannot explain their own code.	Member wrote functional, well-documented code for assigned modules; can explain the logic and OS APIs used in their specific files.	Member wrote complex, highly optimized, and robust code for critical modules (e.g., IPC, sync); demonstrates deep mastery of their specific C implementations.
6.2 Report & Analysis Contribution	Member contributed nothing to the report, or text is plagiarized/AI-generated without effort.	Member only formatted the document or wrote superficial summaries; lacks technical depth in their assigned sections.	Member wrote clear, technically accurate sections of the report; contributed meaningfully to the analysis, Amdahl's Law calculations, or architecture diagrams.	Member authored high-quality, insightful sections of the report; demonstrates exceptional analytical depth, critical thinking, and professional academic writing in their specific parts.
6.3 Teamwork, Communication & Peer Eval	Peer evaluations indicate the member was absent, uncommunicative, or actively hindered the group (free-rider).	Peer evaluations indicate poor communication, missed internal deadlines, or required constant reminders to complete basic tasks.	Peer evaluations indicate reliable communication, met internal deadlines, and collaborated constructively with the team.	Peer evaluations highlight exceptional leadership, proactive communication, helped unblock peers, and significantly elevated the group's overall performance.
6.4 Project Comprehension (Video/Demo)	Member is completely absent from the demo video or cannot answer basic questions about the project.	Member appears in the video but speaks only in memorized, superficial terms; fails to understand the underlying OS concepts when questioned.	Member clearly explains their specific contributions in the video; demonstrates a solid understanding of the OS concepts (threads, IPC, syscalls) related to their work.	Member articulates the entire system architecture flawlessly in the video; can seamlessly connect their specific work to the broader OS concepts and defend design choices.

8. Hardware, Data Generation & Environment Guidelines

Note 1: System Calls vs. Standard I/O:

You must use `open()`, `read()`, `write()`, and `close()`. - *Crucial Detail:* You cannot pass a 1 GB buffer to a single `read()` or `write()` call. The OS will not process it all at once. You **must** implement a loop to read and write the data in smaller chunks (e.g., 4 MB or 16 MB at a time) until the entire 1 GB is transferred.

Note 2: Memory & Data Types:

1 Billion 32-bit integers require approximately **1 GB of RAM**. - **Best Practice:** Use `<stdint.h>` and explicitly define your arrays using `int32_t`. Ensure you are compiling and running on a **64-bit OS** to avoid 32-bit file size limitations (if you are on a 32-bit OS, you must use `O_LARGEFILE` and `lseek64`).

Note 3: Shared Memory is Mandatory for Multiprocessing:

When using `fork()`, the OS uses Copy-on-Write (CoW). If each child modifies its chunk in private memory, the OS will duplicate the 4 GB array per child, crashing your machine. You **must** place the dataset in POSIX Shared Memory (`shm_open` + `mmap` with `MAP_SHARED`) so that all children operate on the **same physical pages**. You will `read()` the file data directly into this `mmap`'ed pointer.

Note 4: Correctness Verification:

After every sort, your program must verify that the output array is correctly sorted (a simple $O(N)$ linear scan). Print PASS or FAIL to the terminal. Do not include the verification time in your benchmark measurements.

Note 5: Compilation Flags:

Ensure your Makefile links the necessary POSIX real-time and threading libraries.

```
CC = gcc
CFLAGS = -Wall -Wextra -O2 -pthread -D_FILE_OFFSET_BITS=64
LDFLAGS = -lrt -pthread

all: parallel_merge_sort

parallel_merge_sort: main.c sort.c merge.c ipc_utils.c file_utils.c
    $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

clean:
    rm -f parallel_merge_sort *.o dataset.bin
```

(Note: `-D_FILE_OFFSET_BITS=64` is added to ensure `open` and `lseek` can handle files larger than 2GB on all architectures).

9. Submission Instructions

- Create a single ZIP file named [GroupID]_[TutorialSection].zip
- The ZIP must contain folder named GroupID_TutorialSection with the following elements:
 - src/ (folder with all .c and .h files)
 - Makefile
 - report.pdf
 - demo_video.mp4 (or a text file with a link to the video)
 - raw_data.csv
 - README.md
- Submit via [**eBwise**] by 5:00 PM on the due date. Late submissions will incur a 20% penalty per day for the next 2 days. After that zero will be awarded and you will fail the subject since this is an alternative to the final exam assessment.